Level 2

Hackathon – Fundamentals of Web Development

STUDENT ATTENDANCE TRACKER

A responsive browser-based application for maintaining student records,
marking daily attendance and monitoring attendance percentages

Student Name	S. MOHAMMED KALEEM
Register Number	36725U18016
Department	B.Sc. Computer Science
ID	astvu36736725u18016
College	Global College of Arts and Science
Date of Submission	31 October 2026

GITHUB LINK – FULL SOURCE CODE (CLICK TO OPEN)

https://github.com/kaleemfreefire9-del/Kaleem.git

Clear step-by-step explanation of the GitHub link:

1. **Click the blue link above** (or copy-paste the URL into Chrome / Edge / Firefox).
2. The GitHub repository page will open in the browser.
3. You will see exactly **three files** that make the complete application:
   - **index.html** – complete page structure and user interface
   - **style.css** – complete styling and responsive design
   - **script.js** – complete JavaScript logic (CRUD, attendance, localStorage, calculations)
4. Click any file name to view the **full source code online**.
5. To download everything: click the green **Code** button → choose **Download ZIP**.
6. Extract the ZIP → open **index.html** in any modern browser. No installation, no server, no extra software needed.

What the repository contains: The complete, ready-to-run front-end application (HTML + CSS + JavaScript). All student and attendance data is stored in the browser's localStorage. There is no backend. Anyone can open the link, view the code, download it, and run the project immediately.

The same three source files are also supplied in the project ZIP uploaded with this report.

1. Problem Statement

In most colleges and schools, attendance is still maintained in paper registers. This traditional method has several practical difficulties:

- Writing attendance every day takes significant time, especially for large classes.
- Searching an older attendance record requires flipping through many pages.
- Calculating attendance percentage manually is slow and can lead to arithmetic mistakes.
- Identifying students with low attendance requires extra effort and is often done only at the end of the semester.
- Paper registers can be lost, damaged or difficult to share with other staff members.

Because of these problems, teachers and academic staff need a simple digital tool that can store student details, record daily attendance status (Present, Absent, Late or Leave), calculate percentages automatically, and quickly highlight students whose attendance falls below a required threshold.

This project solves the above problem by providing a single-page, responsive web application that runs entirely inside the browser. The application prevents duplicate attendance entries for the same student on the same date, updates the dashboard instantly, and keeps all data safely inside the browser's localStorage so that records survive page reloads. No internet connection is required after the files are available on the device.

The application is designed as a learning project under the Naan Mudhalvan / TN Skill Level-2 Hackathon (Fundamentals of Web Development – Oracle). It demonstrates practical use of HTML for structure, CSS for responsive layout, and JavaScript for interactive logic and data storage.

2. Objectives

The overall purpose of the Student Attendance Tracker is to replace a paper register with a clean, interactive browser interface. The specific objectives that guided the development of the application are listed below.

Primary Objectives

- Provide a simple form to add new students with a unique Student ID, full Name and Department name.
- Allow existing student information to be edited at any time and allow student records to be deleted (when a student is deleted, all of that student's attendance history is also removed).
- Record exactly one attendance status per student for any selected calendar date.
- Support four attendance statuses: Present, Absent, Late and Leave.
- Prevent the creation of duplicate attendance entries for the same student on the same date.
- Calculate each student's attendance percentage dynamically using only Present and Absent records (Late and Leave are stored for information but do not affect the percentage).
- Display a live summary dashboard that shows Total Students, Present Today, Absent Today and the Overall Attendance percentage across all students.
- Enable search of students by Student ID, Name or Department and enable filtering of the table by the attendance status of the currently selected date.
- Allow the user to open an individual student's attendance history, view all past dates and remove any single history record if needed.
- Highlight in a different colour those students whose attendance percentage is below a user-configurable minimum threshold (default value is 75%).
- Persist every change immediately into the browser's localStorage so that the data remains available after the page is closed and reopened on the same device.

Achieving these objectives produces a complete, self-contained front-end application that can be demonstrated, tested and used for small-scale attendance management without any server or database installation.

3. Architecture

The Student Attendance Tracker follows a pure client-side architecture. There is no web server, no backend language and no external database. Everything runs inside the user's browser. The application consists of only three files that work together as follows.

Layer	File	Responsibility
Presentation	index.html	Defines the page header, four summary cards, the Add/Edit student form, date & threshold controls, search box, status filter, main student table and the history panel.

Styling	style.css	Defines colours, spacing, card appearance, button styles (Present/Absent/Late/Leave), table layout and responsive media queries for mobile and tablet screens.
Application Logic	script.js	Validates all user input, performs CRUD operations on students, marks and updates attendance, calculates percentages, applies search/filter, renders the UI and saves state to localStorage.
Persistence	localStorage	Stores the entire application state as a JSON object under the key **attendanceTrackerData**. Data survives browser reloads on the same device.

High-level user navigation / application flow

1. The user opens **index.html** in any modern browser (Chrome, Edge, Firefox, etc.).
2. The JavaScript file loads. It reads the previously saved JSON from localStorage. If no data exists, it starts with empty student and attendance arrays.
3. The user can add a new student by filling the form (Student ID, Name, Department) and clicking Add Student. The system checks that the ID is unique.
4. To change an existing student, the user clicks Edit, modifies the fields and clicks Update Student. Cancel is also available.
5. The user selects a date from the date picker and uses the Present / Absent / Late / Leave buttons next to each student to mark attendance for that day.
6. After every change the application validates the action, updates the in-memory state, writes the new JSON to localStorage and immediately re-renders the dashboard cards and the student table.
7. The user can type in the search box, change the status filter, or adjust the minimum threshold percentage; the table updates in real time without a page refresh.
8. Clicking the History button for any student opens a panel that lists all past attendance records for that student. Individual records can be deleted from this panel.

Because the entire application is client-side, the same three files can be placed on a USB drive, shared by email, or hosted on any static web server and will work identically.

4. Practical Implementation

The application was developed using only the three core web technologies taught in the Level-2 Fundamentals of Web Development course: HTML5, CSS3 and vanilla JavaScript. No external frameworks (React, Angular, Vue), no CSS libraries (Bootstrap, Tailwind) and no JavaScript libraries (jQuery, Lodash) were used. All interactive behaviour is written in a single script.js file that uses standard DOM methods, event listeners, localStorage and pure functions for validation and calculation.

Development was carried out in a simple text editor. The three files were kept in one folder so that they can be opened directly by any browser without a build step or package manager. This design matches the expectation of a Level-2 front-end project and makes the code easy for evaluators to inspect.

4.1 Features Implemented

Each of the features required by the problem statement has been implemented and is described below.

- **Add / Edit / Delete Student:** A form accepts Student ID, Name and Department. The system rejects an empty field or a duplicate Student ID. Clicking Edit loads the selected student's data into the form and changes the button text to "Update Student". Deleting a student also removes every attendance record belonging to that student.
- **Mark Attendance by Date:** A date picker sets the working day. Four coloured buttons (Present – green, Absent – red, Late – orange, Leave – grey) appear next to every student. Only one status is allowed per student per date; if a status already exists it is simply overwritten.
- **Dynamic Percentage Calculation:** For each student the percentage is calculated as (number of Present days) ÷ (Present + Absent days) × 100. Late and Leave records are stored and shown in the history but are intentionally excluded from the percentage so that only clear attendance decisions affect the score.
- **Dashboard Summary Cards:** Four cards at the top of the page show Total Students, Present Today, Absent Today and Overall Attendance percentage. The cards update instantly after every change.
- **Search and Status Filter:** A search box filters the table by Student ID, Name or Department as the user types. A drop-down filter can restrict the table to students who are Present, Absent, Late, Leave or Not Marked on the selected date.
- **Attendance History Panel:** Clicking the History button for any student opens a panel that lists every date and status recorded for that student. Any single record can be deleted from this panel; the main table and dashboard are refreshed

automatically.

• **Low-Attendance Highlight:** A numeric threshold input (default 75) controls which students are considered low-attendance. Any student whose percentage is below the threshold is shown with a light red background and a “(Low)” label next to the percentage.

• **Local Storage Persistence:** After every successful operation the complete state (students array + attendance array) is written to localStorage under the key attendanceTrackerData. When the page is reopened the same data is loaded again, so the application works offline after the first visit.

• **Responsive Design:** CSS media queries rearrange the summary cards into two columns on tablets and into a single column on mobile phones. Form fields and buttons also stack vertically on narrow screens so the interface remains usable on a smartphone.

4.2 Full Source Code of the Application

The complete source code of all three files is given below exactly as present in the GitHub repository and project ZIP.

HTML – index.html (complete)

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Student Attendance Tracker</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <header>
    <h1>Student Attendance Tracker</h1>
    <p>Global College of Arts and Science | B.Sc. (Computer Science)</p>
  </header>
  <main>
    <section class="summary" aria-label="Attendance summary">
      <article class="card"><span>Total Students</span><strong id="total">0</strong></article>
      <article class="card"><span>Present Today</span><strong id="present">0</strong></article>
      <article class="card"><span>Absent Today</span><strong id="absent">0</strong></article>
      <article class="card"><span>Overall Attendance</span><strong id="overall">0%</strong></article>
    </section>
    <section class="panel">
      <h2 id="formTitle">Add Student</h2>
      <form id="studentForm">
        <input type="hidden" id="editOriginalId">
        <input id="studentId" placeholder="Student ID (e.g., 36725U18016)" required>
        <input id="studentName" placeholder="Student Name" required>
        <input id="department" placeholder="Department (e.g., B.Sc. CS)" required>
        <button type="submit" id="submitBtn">Add Student</button>
        <button type="button" id="cancelEditBtn" class="btn-secondary hidden">Cancel</button>
      </form>
    </section>
    <section class="panel">
      <h2>Mark Daily Attendance &amp;amp; Records</h2>
      <div class="controls-bar">
        <div class="control-group"><label for="attendanceDate">Date:</label><input id="attendanceDate" type="date" required></div>
        <div class="control-group"><label for="threshold">Min Threshold (%):</label><input id="threshold" type="number" min="0" max="100" va
        <div class="control-group grow"><input id="search" placeholder="Search by name, ID or department..."></div>
        <div class="control-group"><select id="statusFilter"><option value="All">All Statuses</option><option value="Present">Present</optio
      </div>
      <p id="message" role="status"></p>
      <div class="table-wrap"><table><thead><tr><th>Student ID</th><th>Student Name</th><th>Department</th><th>Percentage</th><th>Status (Se
    </section>
    <section id="historyPanel" class="panel hidden">
      <div class="history-header"><h2 id="historyTitle">Attendance History</h2><button type="button" id="closeHistoryBtn" class="btn-seconda
      <div class="table-wrap"><table><thead><tr><th>Date</th><th>Status</th><th>Action</th></tr></thead><tbody id="historyRows"></tbody></ta
    </section>
  </main>
  <script src="script.js"></script>
</body>
</html>
```

CSS – style.css (complete)

```
* { box-sizing: border-box; }
body { margin: 0; font-family: Arial, sans-serif; background: #f3f6fb; color: #263548; }
header { background: #173656; color: white; padding: 24px 5%; }
header h1 { margin: 0 0 8px; }
header p { margin: 0; color: #dce9f8; }
main { max-width: 1200px; margin: 24px auto; padding: 18px; }
.summary { display: grid; grid-template-columns: repeat(4, 1fr); gap: 14px; margin-bottom: 20px; }
.card, .panel { background: white; border: 1px solid #d8e1ed; border-radius: 12px; padding: 18px; }
.card span { display: block; color: #65758b; font-size: 13px; text-transform: uppercase; }
.card strong { display: block; font-size: 28px; margin-top: 10px; color: #1c4e80; }
.panel { margin-bottom: 18px; }
.panel h2 { margin-top: 0; font-size: 18px; color: #173656; }
form, .controls-bar { display: flex; flex-wrap: wrap; gap: 12px; align-items: center; }
.control-group { display: flex; align-items: center; gap: 6px; }
.control-group.grow { flex: 1; }
.control-group.grow input { width: 100%; }
input, select, button { padding: 10px 12px; border: 1px solid #cbd6e4; border-radius: 7px; font-size: 14px; }
input { min-width: 150px; }
input.small-input { min-width: 70px; width: 75px; }
button { background: #2f75b5; color: white; border: 0; cursor: pointer; transition: background 0.2s; }
button:hover { background: #245d91; }
.btn-present { background: #217346; } .btn-present:hover { background: #195935; }
.btn-absent { background: #b42318; } .btn-absent:hover { background: #8c1b12; }
.btn-late { background: #d97706; } .btn-late:hover { background: #b45309; }
.btn-leave { background: #64748b; } .btn-leave:hover { background: #475569; }
.btn-secondary { background: #e2e8f0; color: #1e293b; } .btn-secondary:hover { background: #cbd5e1; }
.btn-danger { background: #fee2e2; color: #b42318; } .btn-danger:hover { background: #fecaca; }
.btn-sm { padding: 6px 10px; font-size: 12px; margin-right: 4px; }
.table-wrap { overflow-x: auto; margin-top: 14px; }
table { width: 100%; border-collapse: collapse; background: white; }
th, td { padding: 12px; text-align: left; border-bottom: 1px solid #e4eaf2; font-size: 14px; }
th { background: #eaf1f9; color: #173656; }
tr.low-row { background-color: #fffbfa; }
.status-present { color: #217346; font-weight: bold; }
.status-absent, .low { color: #b42318; font-weight: bold; }
.status-late { color: #d97706; font-weight: bold; }
.status-leave { color: #64748b; font-weight: bold; }
#message { min-height: 20px; margin: 10px 0 0; font-weight: bold; color: #217346; }
#message.error { color: #b42318; }
.hidden { display: none; }
.history-header { display: flex; justify-content: space-between; align-items: center; }
@media (max-width: 760px) { .summary { grid-template-columns: repeat(2, 1fr); } form, .controls-bar { flex-direction: column; align-items: s
@media (max-width: 420px) { .summary { grid-template-columns: 1fr; } }
```

JavaScript – script.js (complete)

```
const KEY = "attendanceTrackerData";
const state = JSON.parse(localStorage.getItem(KEY) || '{"students": [], "attendance": []}');
const $ = id => document.getElementById(id);
const today = new Date().toISOString().slice(0, 10);
$("attendanceDate").value = today;
let activeHistoryStudentId = null;
function save() { localStorage.setItem(KEY, JSON.stringify(state)); }
function showMessage(text, isError = false) {
  const msgEl = $("message"); msgEl.textContent = text; msgEl.className = isError ? "error" : "";
}
function escapeHTML(value) {
  return String(value).replace(/[&<>"']/g, ch => ({
    "&": "&amp;", "<": "&lt;", ">": "&gt;", '"': "&quot;", "'": "&#39;"
  }[ch]));
}
function saveStudent(originalId, studentId, name, department) {
  studentId = studentId.trim(); name = name.trim(); department = department.trim();
  if (!studentId || !name || !department) throw new Error("All student fields are required.");
  if (originalId) {
    if (originalId !== studentId && state.students.some(s => s.studentId.toLowerCase() === studentId.toLowerCase())) throw new Error("New St
    const student = state.students.find(s => s.studentId === originalId);
    if (student) {
      student.studentId = studentId; student.name = name; student.department = department;
      state.attendance.forEach(a => { if (a.studentId === originalId) a.studentId = studentId; });
    }
  } else {
    if (state.students.some(s => s.studentId.toLowerCase() === studentId.toLowerCase())) throw new Error("Student ID already exists.");
    state.students.push({ studentId, name, department });
  }
  save();
}
function deleteStudent(studentId) {
  if (!confirm(`Delete student ${studentId} and all their attendance records?`)) return;
  state.students = state.students.filter(s => s.studentId !== studentId);
  state.attendance = state.attendance.filter(a => a.studentId !== studentId);
  if (activeHistoryStudentId === studentId) { $("historyPanel").classList.add("hidden"); activeHistoryStudentId = null; }
  save(); showMessage("Student deleted."); render();
}
function startEditStudent(studentId) {
  const student = state.students.find(s => s.studentId === studentId); if (!student) return;
  $("editOriginalId").value = student.studentId; $("studentId").value = student.studentId;
  $("studentName").value = student.name; $("department").value = student.department;
  $("formTitle").textContent = "Edit Student"; $("submitBtn").textContent = "Update Student";
  $("cancelEditBtn").classList.remove("hidden"); $("studentId").focus();
}
function resetForm() {
  $("studentForm").reset(); $("editOriginalId").value = "";
  $("formTitle").textContent = "Add Student"; $("submitBtn").textContent = "Add Student";
  $("cancelEditBtn").classList.add("hidden");
}
function markAttendance(studentId, date, status) {
  if (!date) throw new Error("Please select a valid date.");
  const existing = state.attendance.find(a => a.studentId === studentId && a.date === date);
  if (existing) throw new Error(`Attendance already recorded as '${existing.status}' for ${date}. Delete it from History to re-mark.`);
  state.attendance.push({ studentId, date, status }); save();
}
function percentage(studentId) {
  const rows = state.attendance.filter(a => a.studentId === studentId && ["Present", "Absent"].includes(a.status));
  if (!rows.length) return 0;
  return Math.round(rows.filter(a => a.status === "Present").length / rows.length * 100);
}
function getStatusClass(status) {
  if (status === "Present") return "status-present";
  if (status === "Absent") return "status-absent";
  if (status === "Late") return "status-late";
  if (status === "Leave") return "status-leave";
  return "";
}
function renderHistory(studentId) {
  const student = state.students.find(s => s.studentId === studentId); if (!student) return;
  activeHistoryStudentId = studentId;
  $("historyTitle").textContent = `Attendance History: ${student.name} (${student.studentId})`;
  const records = state.attendance.filter(a => a.studentId === studentId).sort((a,b) => b.date.localeCompare(a.date));
  $("historyRows").innerHTML = !records.length ? `<tr><td colspan="3">No attendance records found.</td></tr>` :
    records.map(r => `<tr><td>${escapeHTML(r.date)}</td><td class="${getStatusClass(r.status)}">${escapeHTML(r.status)}</td><td><button clas
  $("historyPanel").classList.remove("hidden");
}
function render() {
  const date = $("attendanceDate").value, q = $("search").value.toLowerCase().trim();
  const statusFilter = $("statusFilter").value, threshold = Number($("threshold").value) || 75;
  const filteredStudents = state.students.filter(s => {
    const matchesSearch = [s.studentId, s.name, s.department].join(" ").toLowerCase().includes(q);
    const saved = state.attendance.find(a => a.studentId === s.studentId && a.date === date);
    const currentStatus = saved ? saved.status : "Not marked";
    return matchesSearch && (statusFilter === "All" || currentStatus === statusFilter);
  });
  $("studentRows").innerHTML = !filteredStudents.length ? `<tr><td colspan="7">No matching student records found.</td></tr>` :
    filteredStudents.map(s => {
      const saved = state.attendance.find(a => a.studentId === s.studentId && a.date === date);
      const status = saved ? saved.status : "Not marked", pct = percentage(s.studentId);
```

```
      const hasRecords = state.attendance.some(a => a.studentId === s.studentId && ["Present", "Absent"].includes(a.status));
      const isLow = hasRecords && pct < threshold;
      return `<tr class="${isLow ? 'low-row' : ''}">
        <td>${escapeHTML(s.studentId)}</td><td>${escapeHTML(s.name)}</td><td>${escapeHTML(s.department)}</td>
        <td class="${isLow ? 'low' : 'status-present'}">${pct}% ${isLow ? '(Low)' : ''}</td>
        <td class="${getStatusClass(status)}">${escapeHTML(status)}</td>
        <td><button class="btn-present btn-sm" data-action="mark" data-id="${escapeHTML(s.studentId)}" data-status="Present">Present</button
        <button class="btn-absent btn-sm" data-action="mark" data-id="${escapeHTML(s.studentId)}" data-status="Absent">Absent</button>
        <button class="btn-late btn-sm" data-action="mark" data-id="${escapeHTML(s.studentId)}" data-status="Late">Late</button>
        <button class="btn-leave btn-sm" data-action="mark" data-id="${escapeHTML(s.studentId)}" data-status="Leave">Leave</button></td>
        <td><button class="btn-secondary btn-sm" data-action="history" data-id="${escapeHTML(s.studentId)}">History</button>
        <button class="btn-secondary btn-sm" data-action="edit" data-id="${escapeHTML(s.studentId)}">Edit</button>
        <button class="btn-danger btn-sm" data-action="delete" data-id="${escapeHTML(s.studentId)}">Delete</button></td></tr>`;
    }).join("");
  const todays = state.attendance.filter(a => a.date === date);
  $("total").textContent = state.students.length;
  $("present").textContent = todays.filter(a => a.status === "Present").length;
  $("absent").textContent = todays.filter(a => a.status === "Absent").length;
  const percentages = state.students.map(s => percentage(s.studentId));
  $("overall").textContent = percentages.length ? Math.round(percentages.reduce((a,b) => a+b,0)/percentages.length)+"%" : "0%";
  if (activeHistoryStudentId) renderHistory(activeHistoryStudentId);
}
$("studentForm").addEventListener("submit", event => {
  event.preventDefault();
  try {
    const origId = $("editOriginalId").value;
    saveStudent(origId, $("studentId").value, $("studentName").value, $("department").value);
    showMessage(origId ? "Student updated successfully." : "Student added successfully."); resetForm(); render();
  } catch (error) { showMessage(error.message, true); }
});
$("cancelEditBtn").addEventListener("click", resetForm);
$("studentRows").addEventListener("click", event => {
  const button = event.target.closest("button[data-action]"); if (!button) return;
  const { action, id, status } = button.dataset;
  try {
    if (action === "mark") { markAttendance(id, $("attendanceDate").value, status); showMessage(`Marked ${status} for ${id}.`); render(); }
    else if (action === "edit") startEditStudent(id);
    else if (action === "delete") deleteStudent(id);
    else if (action === "history") renderHistory(id);
  } catch (error) { showMessage(error.message, true); }
});
$("historyRows").addEventListener("click", event => {
  const button = event.target.closest("button[data-del-date]"); if (!button) return;
  const { delId, delDate } = button.dataset;
  state.attendance = state.attendance.filter(a => !(a.studentId === delId && a.date === delDate));
  save(); showMessage(`Removed attendance record for ${delDate}.`); render();
});
$("closeHistoryBtn").addEventListener("click", () => { $("historyPanel").classList.add("hidden"); activeHistoryStudentId = null; });
$("search").addEventListener("input", render); $("statusFilter").addEventListener("change", render);
$("threshold").addEventListener("input", render); $("attendanceDate").addEventListener("change", render);
render();
```

4.3 Project Execution – How to Run

1. Open the GitHub link above and click **Code** → **Download ZIP** (or use the ZIP supplied with this report).
2. Extract the folder. Confirm it contains **index.html**, **style.css** and **script.js** in the same directory.
3. Double-click **index.html** (or right-click → Open with Chrome / Edge / Firefox).
4. The application loads instantly. No server, no npm, no build step is required.
5. Add students → choose date → mark Present / Absent / Late / Leave → use Search, Filter, History and Threshold as needed.
6. Data is automatically saved in the browser's localStorage and remains after page reload.

5. Output Screenshots

Screenshots below are taken from the actual running application after loading sample student and attendance data.

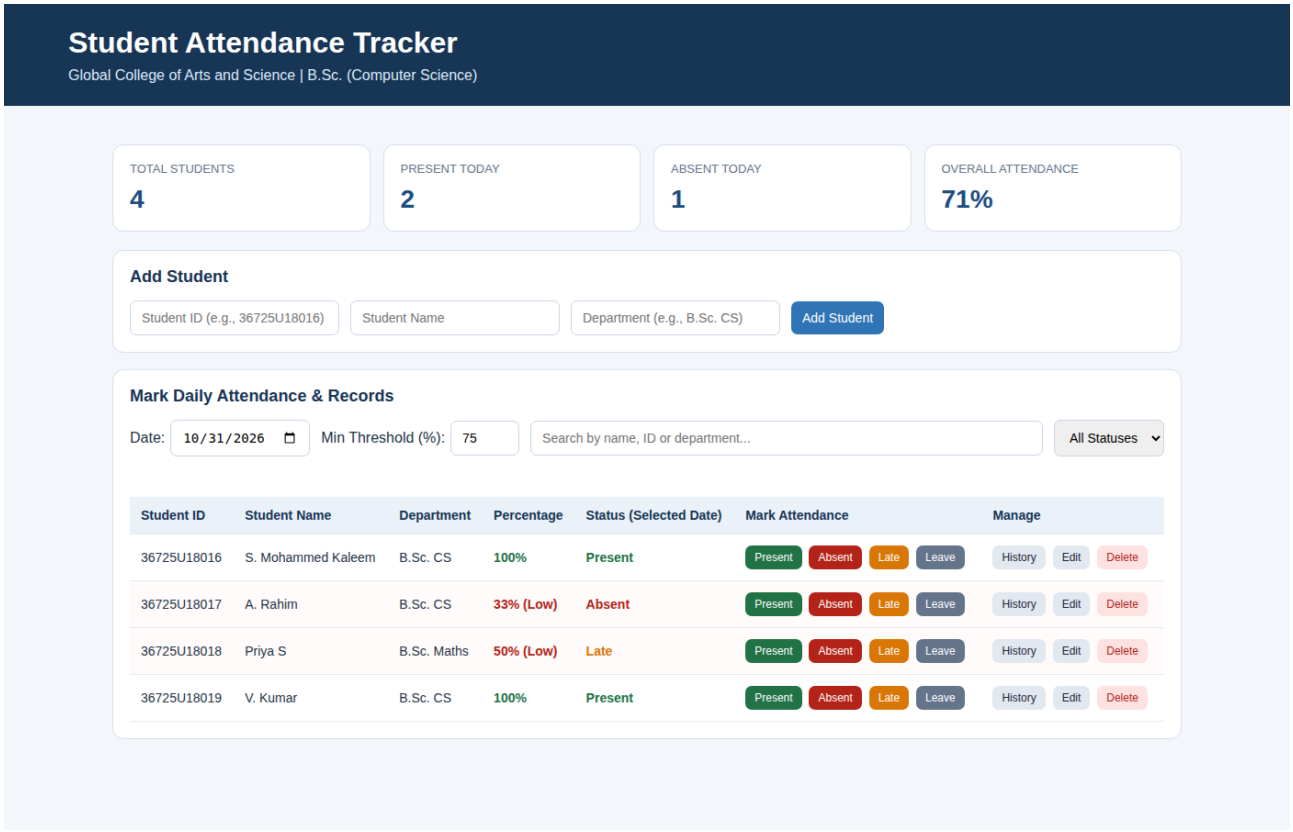


Figure 1: Live dashboard – Total Students = 4, Present Today = 2, Absent Today = 1, Overall Attendance = 71%. Student table shows status buttons, percentage (with Low highlight for students below 75% threshold), History / Edit / Delete actions.

6. Technical Explanation

Technologies used: HTML5 for document structure, CSS3 (Flexbox, CSS Grid and media queries) for visual presentation and responsive layout, and Vanilla JavaScript (ES6+) for all interactive behaviour. No third-party libraries are loaded.

Key programming concepts demonstrated in the code:

- DOM selection and event listeners (including event delegation on the student table).
- localStorage API for reading and writing a JSON object that represents the entire application state.
- Form validation and uniqueness checking before a new student ID is accepted.
- Pure helper functions such as percentage(), escapeHTML() and save() that keep the logic modular and easy to test.
- Dynamic generation of table rows using template strings and careful HTML escaping to avoid XSS risks.
- Real-time filtering of the student list based on search text and status drop-down.
- Responsive design achieved with CSS media queries that change the number of summary-card columns and the layout of form controls.

Attendance percentage formula used in script.js:

Percentage = (Number of Present days) ÷ (Number of Present days + Number of Absent days) × 100.

If a student has no Present or Absent records the percentage is shown as 0%. Late and Leave statuses are stored in the attendance array and appear in the history panel, but they are deliberately excluded from the numerator and denominator so that only definite attendance decisions affect the percentage.

The data model is a single JavaScript object with two arrays: **students** (each item has studentId, name, department) and **attendance** (each item has studentId, date, status). This simple structure is sufficient for the required features and is easy to serialise to JSON for localStorage.

7. Conclusion

The Student Attendance Tracker successfully meets all the requirements of the Level-2 Hackathon problem statement. It provides a complete set of student management operations (add, edit, delete), date-based attendance marking with four statuses, automatic percentage calculation, live dashboard cards, search and status filtering, individual history review, and visual highlighting of students whose attendance falls below a configurable threshold. All data is stored in the browser's localStorage, so the application continues to work after a page reload and does not need a permanent internet connection.

Through the development of this project the following concepts were learned and applied in practice: semantic HTML structure, modern CSS layout techniques, DOM event handling, client-side data persistence with localStorage, input validation, and the clean separation of presentation (HTML/CSS) from application logic (JavaScript). The resulting application is lightweight, easy to run and suitable for demonstration, classroom use or further extension.

8. Future Enhancements

Although the current version fully satisfies the Level-2 requirements, several improvements can be considered for a more production-ready system:

- Add a simple login page and role-based access so that only authorised teachers can mark attendance.
- Connect a backend server (Node.js, PHP or Python) and a real database (MySQL, PostgreSQL or MongoDB) so that data can be shared across multiple devices and users.
- Provide an Export button that generates a CSV or PDF attendance report for any date range or for the whole semester.
- Allow bulk import of a student list from an Excel or CSV file to save time at the beginning of a term.
- Add graphical charts (bar or line) that show monthly or semester-wise attendance trends for the class.
- Permit direct editing of an existing attendance record instead of requiring the user to delete and re-mark it.
- Convert the application into a Progressive Web App (PWA) so that it can be installed on a phone's home screen and used offline more conveniently.
- Add email or SMS notifications that automatically alert students whose attendance drops below the threshold.

GitHub Repository (clickable link):

https://github.com/kaleemfreefire9-del/Kaleem.git

Click the link → view or download the complete source code (index.html, style.css, script.js).

Submitted by: **S. MOHAMMED KALEEM** | Register No: **36725U18016**

Global College of Arts and Science | B.Sc. Computer Science | 31 October 2026